DOC GENIE - PYTHON FUNCTION DOCSTRING GENERATOR

Project Report

Submitted to **Manonmaniam Sundaranar** University in partial fulfillment of the requirements for the award of the degree of

BACHELOR OF SCIENCE IN COMPUTER SCIENCE

NAAN MUDHALVAN COURSE
(Agentic AI Application Developer)

Submitted By

S. ABEDA BEGAM	**23081021802112002**
K. RABIAYATHUL BASHARIYA	**23081021802112025**
S. ROSELIN FATHIMA	**23081021802112028**
P. AFROSE FATHIMA	**20221021506206**

Under the guidance of

Mrs. A. Aaisha Nazleem MCA., M.Phil., Ph.D

Assistant Professor,

DEPARTMENT OF COMPUTER SCIENCE

ANNAI HAJIRA WOMEN'S COLLEGE

(AFFILIATED TO MANONMANIAM SUNDARANAR UNIVERSITY)

TIRUNELVELI - 627005

MARCH– 2026

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
1.	**ABSTRACT**	
2.	**INTRODUCTION**	1
3.	**SYSTEM REQUIREMENTS** 3.1 HARDWARE REQUIREMENTS 3.2 SOFTWARE REQUIREMENTS 3.3 CORE TECHNOLOGIES	2
4.	**SYSTEM DESIGN** 4.1 TECHNICAL ARCHITECTURE 4.2 PROJECT STRUCTURE	3
5.	**PROJECT WORKFLOW**	5
6.	**ALGORITHM**	7
7.	**IMPLEMENTATION** 7.1 CODING 7.2 OUTPUT	8
8.	**CONCLUSION**	17

1. ABSTRACT

Doc-Genie is an intelligent Python documentation platform developed to automatically generate structured and professional docstrings for Python functions. Writing documentation manually is often time-consuming, inconsistent, and frequently neglected by developers, which results in poorly documented software projects and reduced maintainability. The primary objective of Doc-Genie is to simplify and automate the documentation process by analyzing Python code and generating meaningful docstrings in standard documentation formats. The system uses Abstract Syntax Tree (AST) based code analysis techniques to understand the structure of Python programs. By parsing the source code, the platform identifies function definitions, parameters, return values, and other structural components. Based on this analysis, the system automatically generates descriptive docstrings that follow widely accepted documentation standards such as Google style and NumPy style formats. This ensures that the generated documentation remains clear, structured, and compatible with modern development environments. Doc-Genie also provides a user-friendly web interface built using the Gradio framework. Through this interface, users can easily paste Python code or upload Python files for analysis. Once the code is processed, the system generates well-formatted documentation that can be viewed instantly within the interface. In addition, the platform supports exporting the generated documentation into formats such as PDF and TXT, enabling easy sharing and integration with existing software documentation workflows. By automating the documentation process, Doc-Genie significantly reduces manual effort, improves documentation consistency, and enhances overall developer productivity. The system demonstrates how intelligent code analysis techniques can transform traditional documentation practices into a faster, more reliable, and efficient workflow for modern Python development. Furthermore, the system helps programmers quickly understand the structure and functionality of Python code by generating clear and organized documentation. This reduces the effort required for maintaining large codebases and improves overall code readability. The platform also promotes good coding practices by encouraging developers to include proper documentation in their software projects. By integrating automated analysis and documentation generation, Doc-Genie demonstrates how intelligent tools can enhance software development efficiency. The system can also be extended in the future with additional features such as support for more documentation styles, integration with version control systems, and advanced code analysis capabilities. Overall, Doc-Genie serves as a practical solution for modern software development by making Python documentation faster, easier, and more reliable.

2. INTRODUCTION

Doc-Genie is an intelligent documentation generation platform developed to simplify the process of writing docstrings for Python functions. In modern software development, proper documentation plays a critical role in improving code readability, maintainability, and collaboration among developers. However, writing documentation manually is often time-consuming and inconsistent, which leads many developers to skip this important step.

The Doc-Genie system addresses this challenge by automatically generating structured and professional docstrings for Python functions. The system analyzes Python source code using Abstract Syntax Tree (AST) based code analysis techniques. By parsing the structure of the code, the platform identifies function definitions, parameters, return statements, and other relevant elements required to generate meaningful documentation.

Doc-Genie supports widely used documentation formats such as Google style and NumPy style docstrings, which are commonly used in professional Python development environments. The platform provides a simple and user-friendly interface built with the Gradio framework, allowing users to either paste Python code directly or upload Python files for automatic documentation generation.

In addition, the generated documentation can be exported in multiple formats such as PDF and TXT, enabling developers to easily integrate the documentation into their existing projects. By automating the documentation process, Doc-Genie improves developer productivity, ensures documentation consistency, and promotes better coding practices.

Furthermore, the system is designed to enhance the overall developer experience by providing additional features such as file upload support, real-time output status, and easy sharing of generated documentation. These features allow developers to quickly generate, download, and distribute documentation without interrupting their development workflow. By combining automated code analysis with an interactive interface, Doc-Genie serves as a practical tool that supports efficient documentation practices in modern software development environments.

Moreover, Doc-Genie is designed to support both beginners and experienced developers by simplifying the documentation process. It helps reduce manual effort while ensuring that the generated documentation follows a clear and structured format.

3. SYSTEM REQUIREMENTS

3.1 HARDWARE REQUIREMENTS:

- **Processor:** 12th Gen Intel(R) Core(TM) i5-1235U (1.30 GHz)
- **RAM:** 8.00 GB
- **Storage:** 477 GB
- **Display:** Minimum 1024x768 resolution for interface usability

3.2 SOFTWARE REQUIREMENTS:

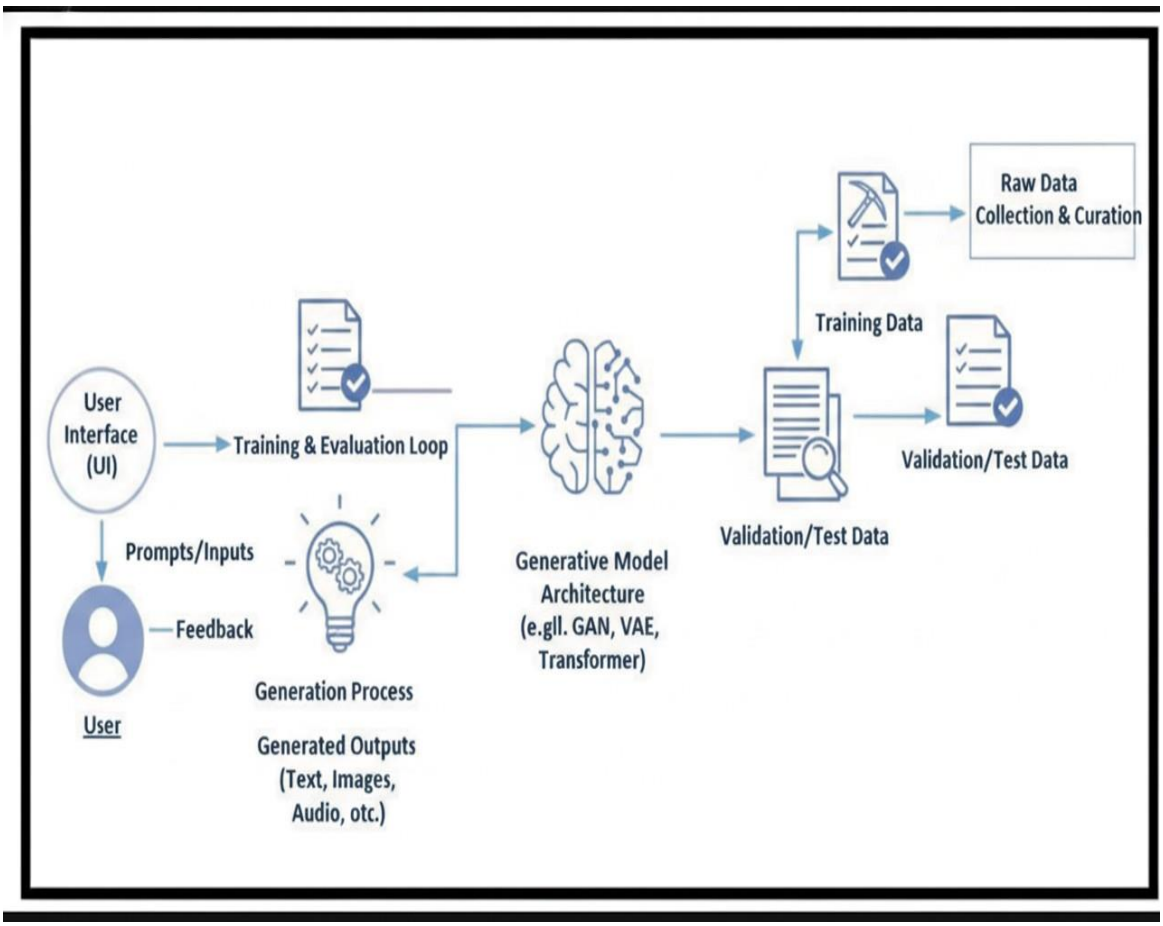
- **Operating System:** Windows 11 pro
- **Programming Language:** Python (Version 3.8 or above)
- **Development Environment:** Google Colab
- **Gradio** – Used to build the user interface for interacting with the system
- **AST (Abstract Syntax Tree)** – Used for analyzing Python code structure
- **ReportLab** – Used for generating PDF documentation
- **Requests** – Used for handling web based sharing features
- **Package Manager:** pip (Python Package Installer)
- **Web Browser:** Google Chrome / Microsoft Edge / Mozilla Firefox

3.3 CORE TECHNOLOGIES:

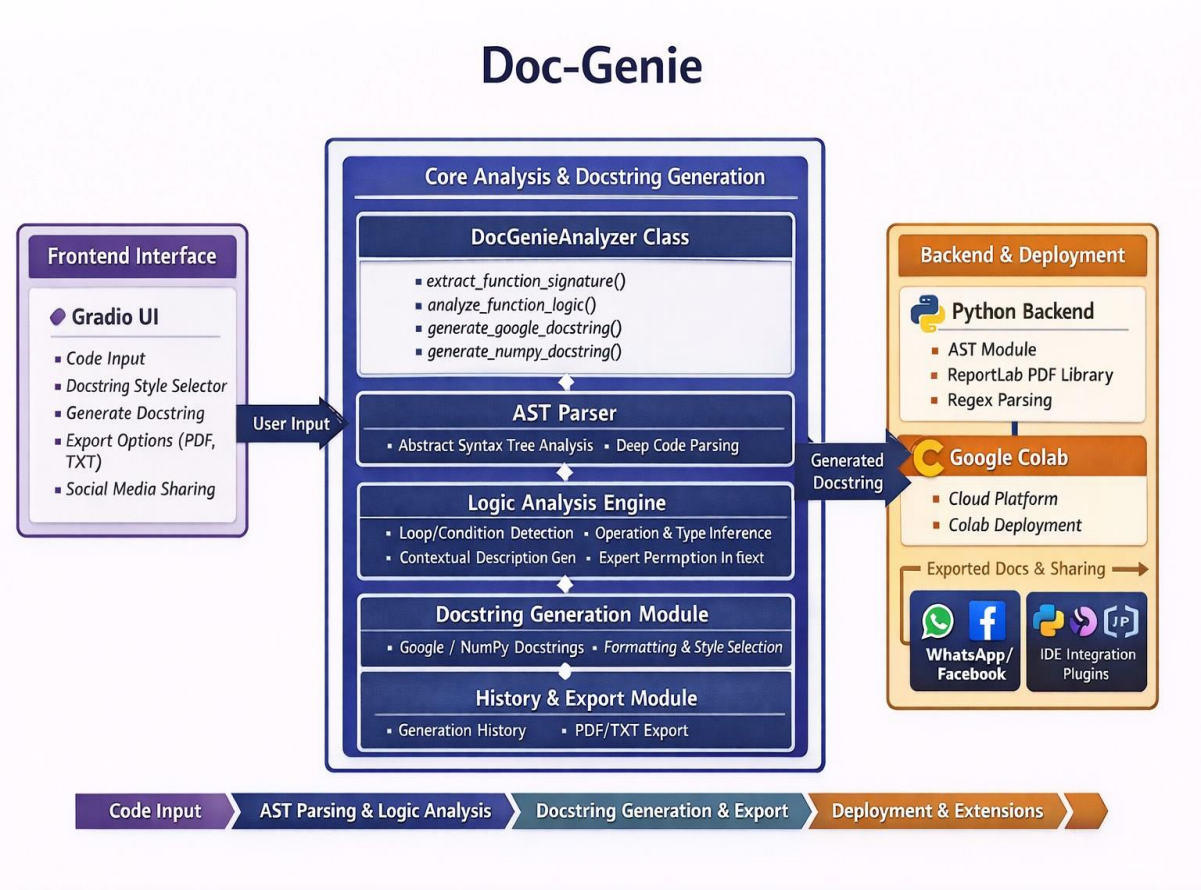
- **Python 3.8+:** Core language with AST module for code analysis
- **Gradio 4.0+:** Interactive web interface framework
- **AST (Abstract Syntax Tree):** Python's built-in code parsing library
- **ReportLab:** PDF generation for comprehensive documentation reports
- **Regular Expressions:** Pattern matching for code analysis
- **Google Colab:** Cloud-based development and deployment environment ·
- **CSS3:** Modern web standards for interface styling

4. SYSTEM DESIGN

4.1. TECHNICAL ARCHITECTURE:



4.2. PROJECT STRUCTURE



5. PROJECT WORKFLOW

The workflow of the Doc-Genie system explains the process followed to generate automatic documentation for Python functions.

Step 1: Environment Setup

The development environment is prepared by installing all the required libraries and dependencies. The main libraries used in this project include Gradio for the user interface, AST module for Python code analysis, and ReportLab for generating PDF documentation.

Step 2: User Input

The user provides Python code to the system in two possible ways:

- Pasting Python code directly into the code input box
- Uploading a Python (.py) file using the file upload option

Step 3: File Upload Processing

If the user uploads a file, the system reads the file content and displays an Upload Status message indicating whether the file was successfully uploaded or if any error occurred.

Step 4: Code Parsing

The system processes the provided Python code and parses it using the Abstract Syntax Tree (AST) module. This helps the system understand the structure of the code.

Step 5: Function Extraction

The system identifies all the function definitions present in the Python code. It extracts important details such as:

- Function name
- Function parameters
- Default parameter values
- Return statements

Step 6: Code Analysis

After extracting the function details, the system analyzes the internal structure of the function. It detects logical components such as loops, conditional statements, and mathematical operations to better understand the function behavior.

Step 7: Docstring Generation

Based on the extracted information, the system automatically generates structured docstrings. The user can select the preferred documentation format such as:

- Google Style Docstring
- NumPy Style Docstring

Step 8: Display Generated Documentation

The generated documentation is displayed in the Generated Result Box, where users can view the updated Python code along with the inserted docstrings.

Step 9: Generated Report Status

After successful generation of the documentation, the system displays a Generated Report Status message to inform the user that the documentation has been generated successfully.

Step 10: Copy and Use Documentation

Users can copy the generated documentation directly from the result box and paste it into their source code or development environment.

Step 11: Export Documentation

The system provides options to export the generated documentation in different formats such as:

- PDF
- TXT

These files can be downloaded and shared easily.

Step 12: Share Documentation

The system also provides a sharing feature that allows users to generate shareable links for the generated documentation. Users can easily share the documentation through platforms such as **WhatsApp and Facebook**, enabling quick collaboration and communication with other developers or team members.

Step 13: End Process

The documentation generation process is completed, and the user can repeat the process for additional Python code.

6. ALGORITHM

Step 1: Start

Step 2: Accept Python code input from the user (paste code or upload file)

Step 3: If a file is uploaded

Display Upload Status

Step 4: Read the Python source code

Step 5: Parse the code using the AST module

Step 6: Identify function definitions in the code

Step 7: Extract important information such as

- Function name
- Parameters
- Return values

Step 8: Analyze the extracted data

Step 9: Generate a structured docstring based on the selected style (Google or NumPy)

Step 10: Insert the generated docstring into the function

Step 11: Display the generated documentation in the Generated Result Box

Step 12: Display Generated Report Status

Step 13: Allow users to copy the generated documentation

Step 14: Provide options to export or share the documentation (PDF / TXT)

Step 15: Stop

7. IMPLEMENTATION

7.1.CODING

1. DocGenieAnalyzer Class (Core Analysis Module)

```
class DocGenieAnalyzer:
    """Analyze Python functions and generate professional docstrings"""

    @staticmethod
    def extract_function_signature(code):
        tree = ast.parse(code)
        func_def = tree.body[0]

        func_name = func_def.name
        params = []

        for arg in func_def.args.args:
            param_type = "Any"
            if arg.annotation:
                param_type = ast.unparse(arg.annotation)
            params.append({
                'name': arg.arg,
                'type': param_type
            })

        return_type = "Any"
        if func_def.returns:
            return_type = ast.unparse(func_def.returns)

        return {
            'name': func_name,
            'params': params,
            'return_type': return_type
        }, func_def
```

2. Function Logic Analysis

```
@staticmethod
def analyze_function_logic(func_def, code):
    analysis = {
        'has_loops': False,
        'has_conditions': False,
        'has_return': False
    }

    for node in ast.walk(func_def):
        if isinstance(node, ast.For) or isinstance(node, ast.While):
            analysis['has_loops'] = True
        elif isinstance(node, ast.If):
            analysis['has_conditions'] = True
        elif isinstance(node, ast.Return):
            analysis['has_return'] = True
    return analysis
```

3. Google Style Docstring Generator

```
def generate_google_docstring(signature, analysis, code):
    docstring = f'    """Executes {signature["name"]} operation.\n'

    if signature['params']:
        docstring += "\n    Args:\n"
        for param in signature['params']:
            docstring += f"        {param['name']} ({param['type']}): Parameter description.\n"

    docstring += f"\n    Returns:\n        {signature['return_type']}: Result of computation.\n"
    docstring += '    """\n'

    return docstring
```

4. NumPy Style Docstring Generator

```
def generate_numpy_docstring(signature, analysis, code):
    docstring = f'    """Executes {signature["name"]} operation.\n'

    if signature['params']:
        docstring += "\n    Parameters\n    ----------\n"
        for param in signature['params']:
            docstring += f"    {param['name']} : {param['type']}\n"

    docstring += f"\n    Returns\n    -------\n    {signature['return_type']}\n"
    docstring += '    """\n'

    return docstring
```

5. File Upload Processing

```
def process_upload(file):
    if file is None:
        return "", "No file uploaded"

    with open(file.name, "r", encoding="utf-8") as f:
        code = f.read()

    return code, "File uploaded successfully"
```

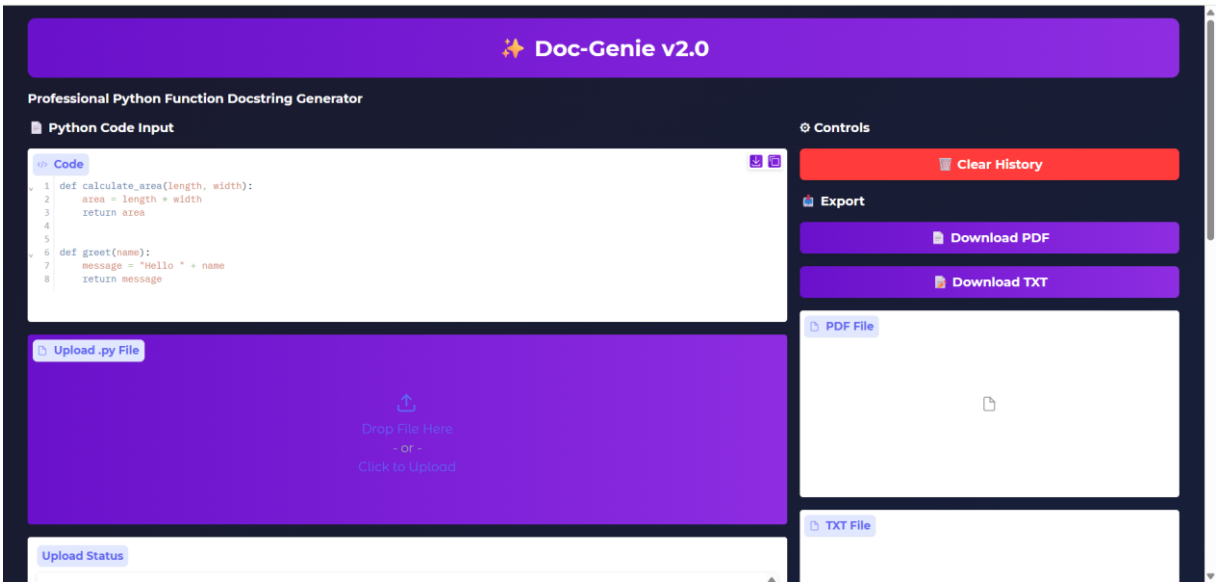
6. User Interface using Gradio

```
with gr.Blocks(title="Doc-Genie") as demo:

    code_input = gr.Code(language="python", label="Code")

    docstring_style = gr.Radio(["google", "numpy"], value="google")

    generate_btn = gr.Button("Generate Docstring")

    code_output = gr.Code(language="python", label="Output")

    generate_btn.click(
        generate_docstring,
        inputs=[code_input, docstring_style],
        outputs=[code_output]
    )
```

7.2. OUTPUT

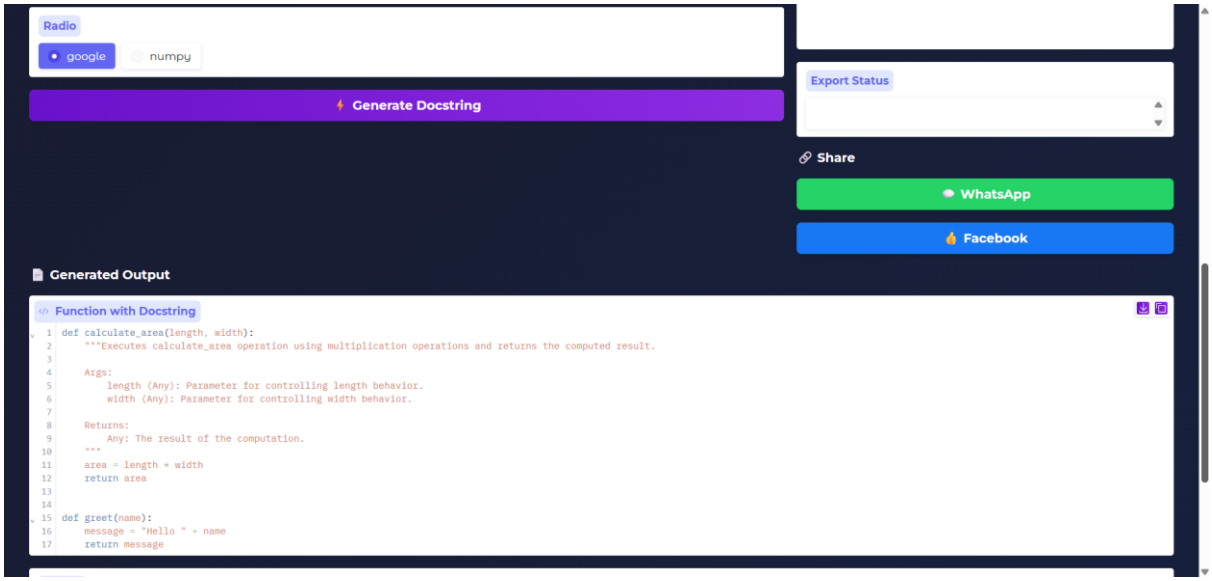
PYTHON CODE INPUT IN THE INTERFACE

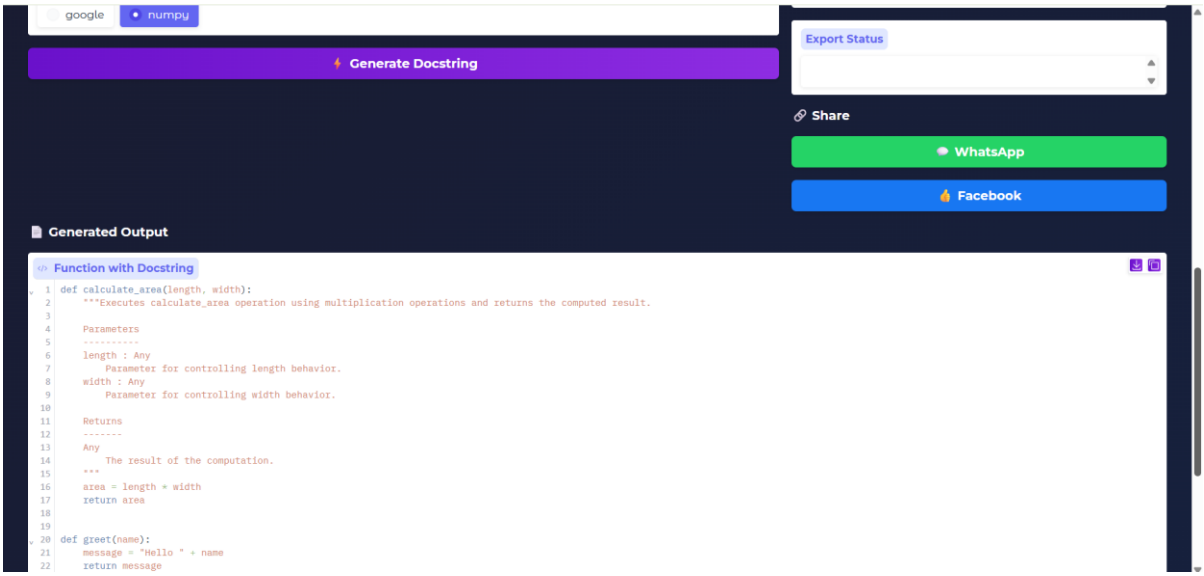
This screenshot shows the code input section where users can paste Python code directly. The system analyzes the entered code and prepares it for docstring generation.



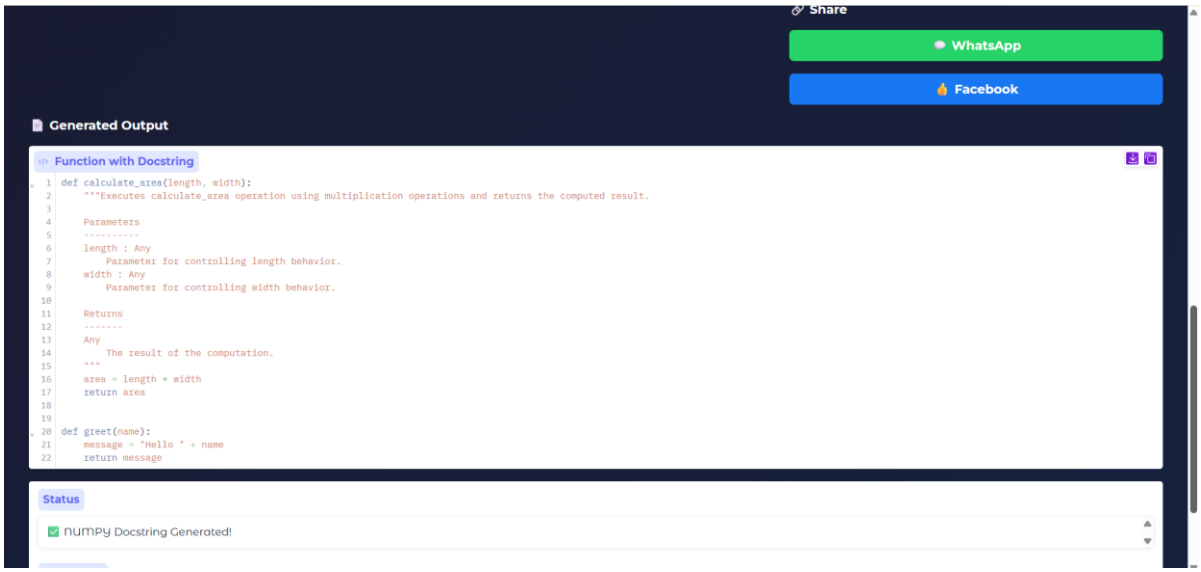
DOCSTRING GENERATED IN GOOGLE STYLE FORMAT

This screenshot shows the feature that allows users to generate docstrings in both Google style and NumPy style formats. After generation, the system also displays a status message indicating that the documentation was successfully created.




google
numpy
Generate Docstring
Export Status
Share
WhatsApp
Facebook
Generated Output
Function with Docstring
def calculate_area(length, width):
"""Executes calculate_area operation using multiplication operations and returns the computed result.
Parameters
---
length : Any
Parameter for controlling length behavior.
width : Any
Parameter for controlling width behavior.
Returns

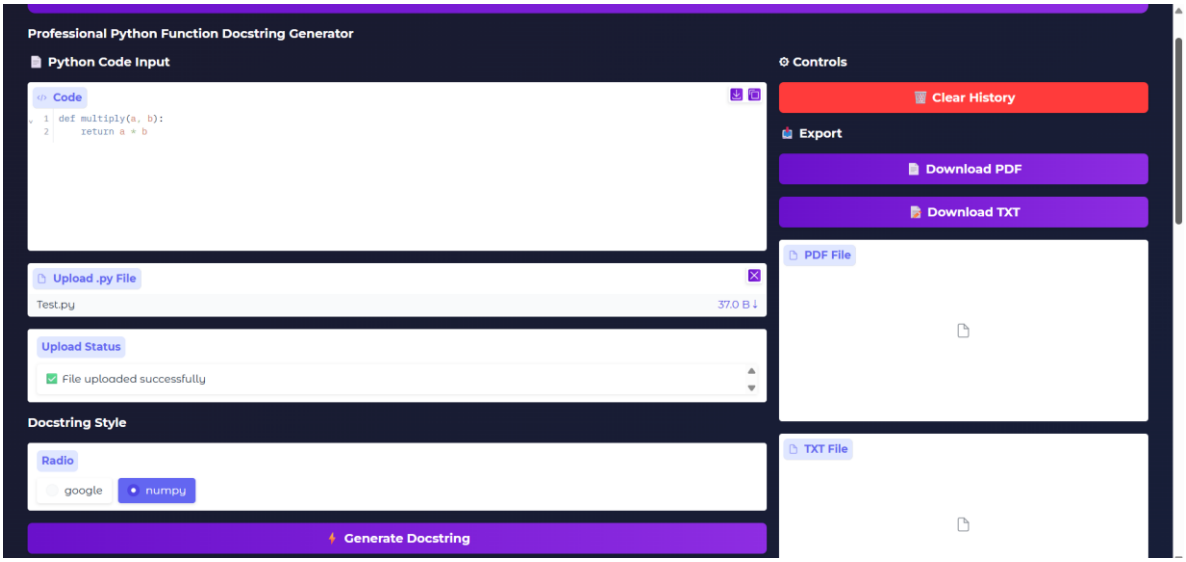
Any
The result of the computation.
"""
area = length * width
return area
def greet(name):
message = "Hello " + name
return message

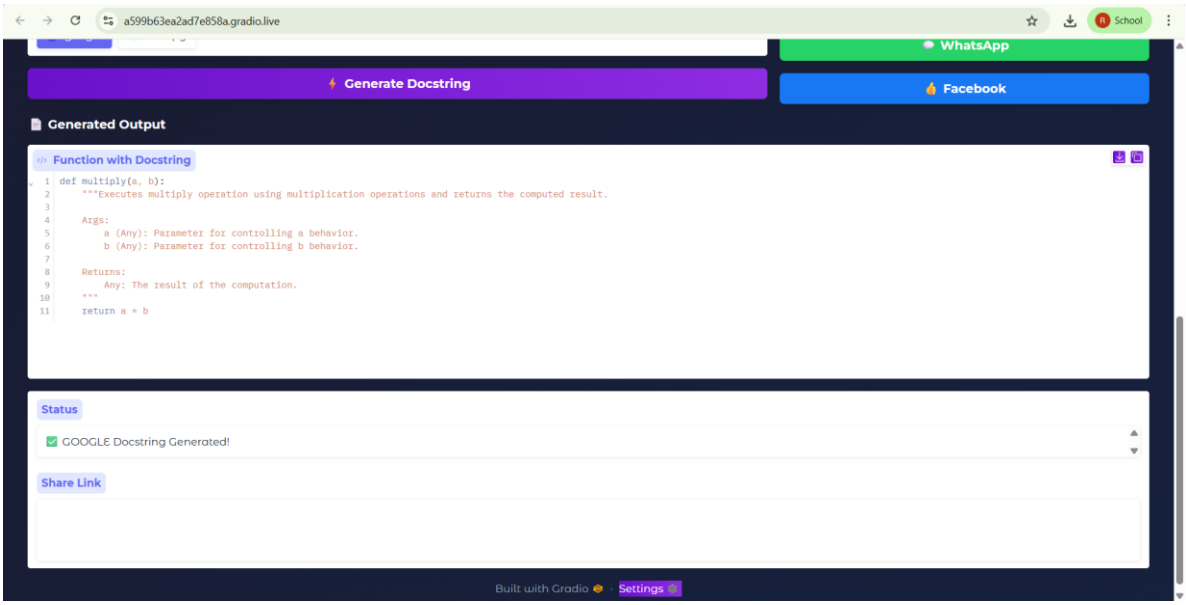

Share
WhatsApp
Facebook
Generated Output
Function with Docstring
def calculate_area(length, width):
"""Executes calculate_area operation using multiplication operations and returns the computed result.
Parameters
---
length : Any
Parameter for controlling length behavior.
width : Any
Parameter for controlling width behavior.
Returns

Any
The result of the computation.
"""
area = length * width
return area
def greet(name):
message = "Hello " + name
return message
Status
numpy Docstring Generated!

UPLOADING PYTHON FILE

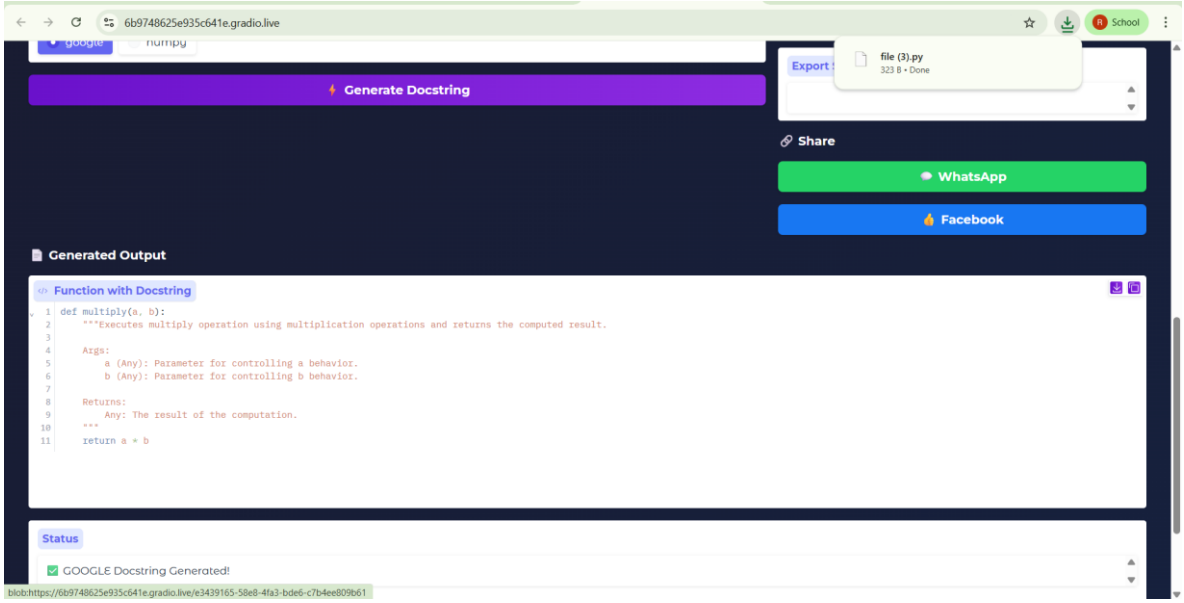
This screenshot shows the file upload option where users can upload a Python (.py) file. The system reads the uploaded file and automatically generates docstrings for the functions in the file.





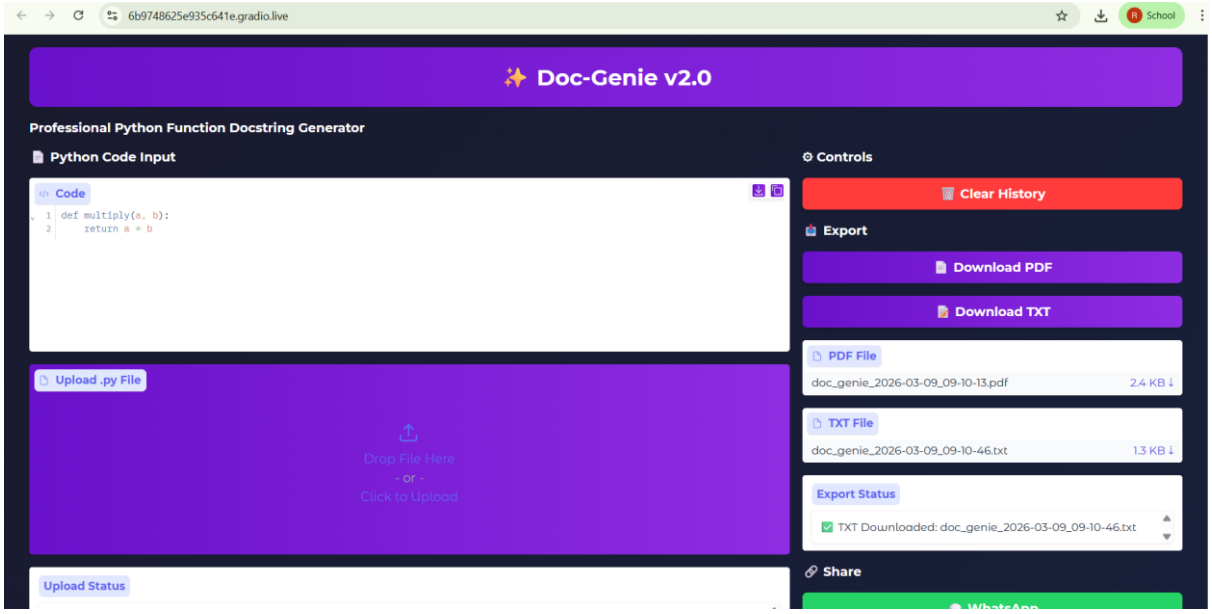
DOWNLOADING GENERATED RESULT USING DOWNLOAD ICON

This screenshot shows the download option available in the interface. Users can click the download icon to save the generated documentation.



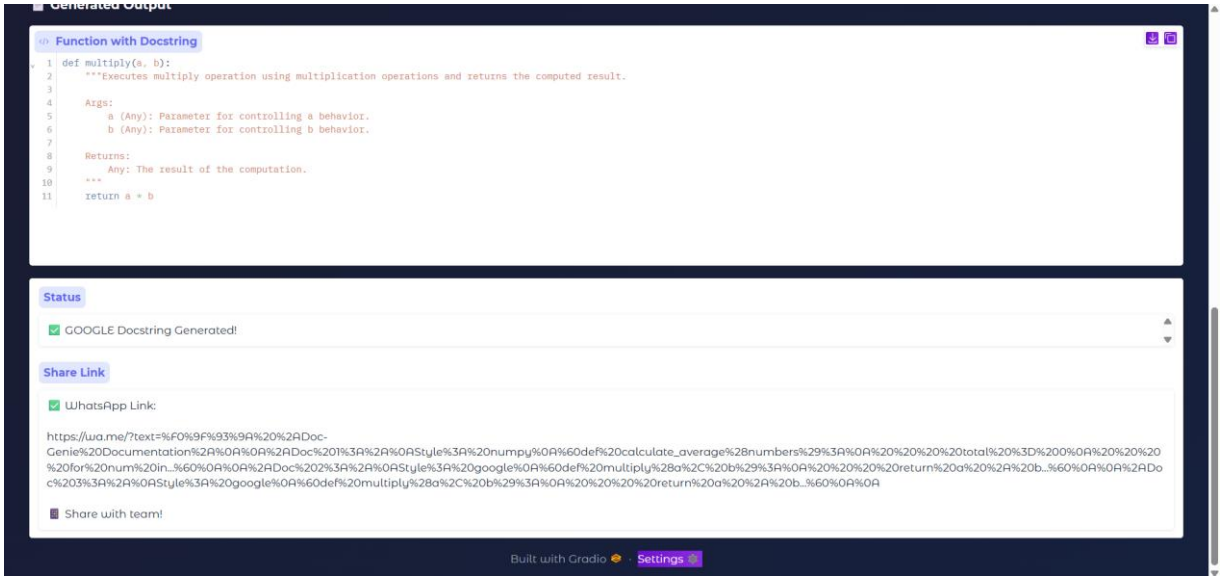
EXPORTING GENERATED DOCSTRING AS PDF AND TEXT FILE

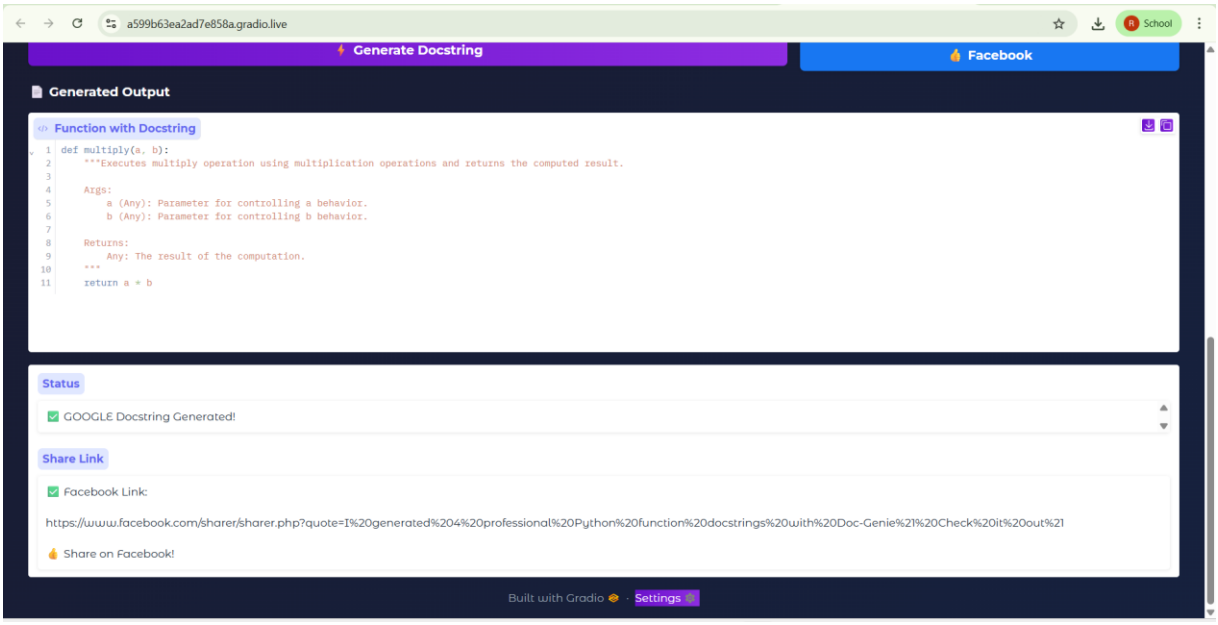
This screenshot shows the export feature of the system. Users can export the generated documentation as PDF or TXT files for easy storage and sharing.



SHARING GENERATED DOCUMENTATION VIA WHATSAPP AND FACEBOOK

This screenshot shows the sharing feature where users can share the generated documentation through WhatsApp and Facebook links.





8. CONCLUSION

The Doc-Genie system was successfully developed to automate the process of generating documentation for Python functions. The main objective of this project was to reduce the manual effort involved in writing docstrings and to improve the quality and consistency of software documentation.

The system analyzes Python source code using the Abstract Syntax Tree (AST) module to identify important elements such as function names, parameters, return values, and internal logic structures. Based on this analysis, the system automatically generates structured docstrings in commonly used formats such as Google style and NumPy style.

The project also provides a user-friendly interface developed using the Gradio framework, allowing users to easily input Python code or upload Python files for analysis. The generated documentation is displayed clearly in the output section, and users can copy the documentation or export it in formats such as PDF and TXT for further use.

Overall, the Doc-Genie system demonstrates how automation can simplify the documentation process in software development. By reducing the time required to write documentation manually, the system helps developers maintain better code quality and improves project maintainability.

Furthermore, the Doc-Genie system encourages developers to follow better documentation practices by providing a quick and efficient way to generate structured docstrings. The platform not only saves development time but also helps maintain consistency across different projects and teams. By integrating code analysis with an interactive interface, the system demonstrates how intelligent tools can support modern software development workflows. In the future, the system can be enhanced by adding support for more documentation formats, integration with development environments, and advanced code analysis features.